

Instituto Politécnico de Viana do Castelo

COMMOV

Sistema de Gestão de Projetos

Relatório de Projeto

Bruno Rosário - 26207 ; Gonçalo Marques - 31415 ; Rafael Monteiro - 27692
/ 3A - Grupo G

Ano letivo 2025/2026

1 de maio de 2026

Conteúdo

1	Introdução	2
2	Objetivos do Projeto	2
3	Estado Atual do Desenvolvimento	3
3.1	Resumo do Backend	3
3.2	Endpoints Disponíveis (exemplos)	3
4	Tecnologias Utilizadas	4
5	Requisitos de Software	4
5.1	Requisitos Funcionais	4
5.2	Requisitos Não Funcionais	5
6	Modelo de Dados	6
6.1	Descrição das Principais Tabelas	8
7	UX e Design	8
7.1	Mockups	8
8	Gestão de Projeto	10
8.1	Evidência de Utilização de Git	11
8.2	Evidência de Utilização de uma Plataforma de Gestão de Projeto	13
9	Conclusão	14

1 Introdução

O presente relatório descreve o estado atual do desenvolvimento do projeto, uma aplicação orientada para a gestão de projetos, tarefas e utilizadores. O sistema pretende permitir que diferentes perfis de utilizador possam acompanhar projetos, gerir tarefas, registar progresso e consultar informação relevante de forma organizada.

Nesta fase, o trabalho desenvolvido centra-se sobretudo na construção da base técnica do projeto, principalmente ao nível do *backend*. O repositório já possui uma estrutura inicial em Go, ligação a uma base de dados PostgreSQL, migrações, repositórios de acesso aos dados e alguns endpoints HTTP funcionais.

Este documento apresenta os objetivos do projeto, o âmbito da aplicação, as tecnologias utilizadas, os requisitos funcionais e não funcionais, uma descrição resumida do código desenvolvido e o modelo de dados da aplicação.

2 Objetivos do Projeto

O principal objetivo do projeto é desenvolver uma aplicação que facilite a gestão de projetos e tarefas, permitindo que administradores, gestores de projeto e utilizadores tenham acesso às funcionalidades adequadas ao seu perfil.

Os objetivos específicos do projeto são:

- Permitir a criação e gestão de contas de utilizador;
- Suportar diferentes perfis de acesso, nomeadamente administrador, gestor de projeto e utilizador;
- Permitir a criação, edição e gestão de projetos;
- Permitir a associação de utilizadores e gestores aos projetos;
- Permitir a criação, atribuição e acompanhamento de tarefas;
- Permitir o registo de progresso, tempo despendido, observações e fotografias associadas às tarefas;
- Disponibilizar dashboards e visualizações que facilitem o acompanhamento do trabalho;
- Permitir a exportação de estatísticas por utilizador, projeto ou tarefa.

3 Estado Atual do Desenvolvimento

Até ao momento, o repositório contém sobretudo a base de um **backend em Go** com uma **API HTTP** e persistência em **PostgreSQL**.

A aplicação já possui uma estrutura organizada por camadas, separando as entidades de domínio, os repositórios de acesso à base de dados, os serviços e as rotas HTTP. Esta organização facilita a manutenção do código e permite evoluir o projeto de forma mais controlada.

Atualmente, já se encontra implementado um endpoint de saúde da aplicação e um endpoint para criação de utilizadores. Embora o domínio de projetos e tarefas já esteja representado na base de dados e nos repositórios, ainda não existem serviços nem rotas HTTP para gerir essas entidades através da API.

3.1 Resumo do Backend

O arranque da aplicação é feito a partir do ficheiro `backend/cmd/main.go`. De forma resumida, o backend executa os seguintes passos:

1. Carrega as configurações da aplicação através de variáveis de ambiente;
2. Estabelece a ligação à base de dados PostgreSQL;
3. Inicializa o repositório e o serviço de utilizadores;
4. Regista e inicia as rotas HTTP disponíveis;

3.2 Endpoints Disponíveis (exemplos)

Tabela 1: Endpoints atualmente disponíveis

Método	Endpoint	Descrição
GET	<code>/health</code>	Verifica se o servidor está ativo, devolvendo o estado da aplicação.
POST	<code>/users</code>	Permite criar um novo utilizador no sistema.

4 Tecnologias Utilizadas

As principais tecnologias utilizadas ou previstas no projeto são:

- **Kotlin Jetpack Compose**: utilizado para construção do aplicação móvel;
- **Go**: linguagem utilizada no desenvolvimento do backend;
- **PostgreSQL**: sistema de gestão de base de dados relacional;
- **GORM**: biblioteca utilizada para interação entre o código Go e a base de dados;
- **Goose**: ferramenta utilizada para gerir migrações da base de dados;
- **Chi**: router HTTP utilizado na API;
- **Docker**: utilizado para contentorização da aplicação;

Nesta fase, o frontend ainda não se encontra implementado no repositório analisado, existindo apenas um ficheiro placeholder.

5 Requisitos de Software

5.1 Requisitos Funcionais

Tabela 2: Requisitos funcionais

ID	Descrição
RF01	O sistema deve apresentar intro sliders ao utilizador.
RF02	O sistema deve permitir a criação de conta.
RF03	O sistema deve permitir o início de sessão.
RF04	O sistema deve permitir a gestão do perfil do utilizador, incluindo nome, username, email, fotografia e password.
RF05	O sistema deve suportar os perfis de Administrador, Gestor de Projeto e Utilizador.
RF06	O administrador deve conseguir criar, editar e remover projetos.
RF07	O administrador deve conseguir criar, alterar e remover perfis de utilizadores e gestores de projeto.
RF08	O administrador deve conseguir associar um gestor de projeto a um projeto.
RF09	O gestor de projeto deve conseguir associar tarefas a cada projeto.

ID	Descrição
RF10	O gestor de projeto deve conseguir associar utilizadores a projetos e às respetivas tarefas.
RF11	O gestor de projeto deve conseguir visualizar tarefas concluídas e por concluir de determinado projeto.
RF12	O gestor de projeto deve conseguir concluir projetos e avaliar a performance dos utilizadores.
RF13	O gestor de projeto deve conseguir definir e gerir roles dos utilizadores associados ao projeto.
RF14	O gestor de projeto deve conseguir atribuir um tempo estimado a cada tarefa.
RF15	O sistema deve disponibilizar dashboards para acompanhamento de projetos e tarefas.
RF16	O utilizador deve conseguir visualizar a lista de tarefas por realizar.
RF17	O utilizador deve conseguir consultar o histórico de tarefas concluídas.
RF18	O utilizador deve conseguir registar a execução de uma tarefa, incluindo data, local, taxa de conclusão e tempo despendido.
RF19	O utilizador deve conseguir associar observações e fotografias às tarefas.
RF20	O utilizador deve conseguir dar uma tarefa como concluída.
RF21	O sistema deve permitir a visualização de tarefas em formato Kanban.
RF22	O sistema deve permitir pesquisa por nome de projeto, utilizador e tarefa.
RF23	O sistema deve permitir a exportação de estatísticas por utilizador, projeto ou tarefa, considerando filtros como período e estado.

5.2 Requisitos Não Funcionais

Tabela 3: Requisitos não funcionais

ID	Descrição
RNF01	O sistema deve garantir controlo de acesso com base no perfil do utilizador.
RNF02	A aplicação deve apresentar uma interface simples, intuitiva e consistente.

ID	Descrição
RNF03	O sistema deve apresentar mensagens de erro claras e compreensíveis.
RNF04	O código deve estar organizado por camadas, facilitando manutenção e evolução futura.
RNF05	A base de dados deve garantir integridade entre utilizadores, projetos e tarefas através de chaves estrangeiras.
RNF06	A aplicação deve poder ser executada em ambiente contentorizado com Docker.
RNF07	A API deve disponibilizar respostas em formato JSON.
RNF08	O sistema deve permitir evolução futura para autenticação, autorização e proteção adequada das passwords.
RNF09	O sistema deve ser preparado para suportar novas rotas e serviços sem alterar drasticamente a estrutura existente.
RNF10	O projeto deve possuir uma estrutura organizada que facilite o desenvolvimento em equipa.

6 Modelo de Dados

A base de dados do projeto foi estruturada para suportar os principais elementos da aplicação: utilizadores, projetos, tarefas e associação entre utilizadores e projetos.

A tabela **Users** guarda os dados principais dos utilizadores, incluindo nome, username, email, password, fotografia, perfil e estado da conta. A tabela **Projects** representa os projetos existentes no sistema, armazenando informação como nome, descrição, estado, datas e referências ao gestor e ao utilizador que criou o projeto.

A tabela **ProjectUsers** funciona como tabela intermédia entre utilizadores e projetos, permitindo associar vários utilizadores ao mesmo projeto. Por fim, a tabela **Tasks** representa as tarefas associadas a projetos e utilizadores, incluindo estado, datas, tempo estimado, tempo gasto, percentagem de conclusão, local, observações e fotografia.

Este modelo permite representar uma estrutura em que um projeto pode ter vários utilizadores associados e várias tarefas. Também permite que cada tarefa esteja ligada a um projeto e a um utilizador responsável.

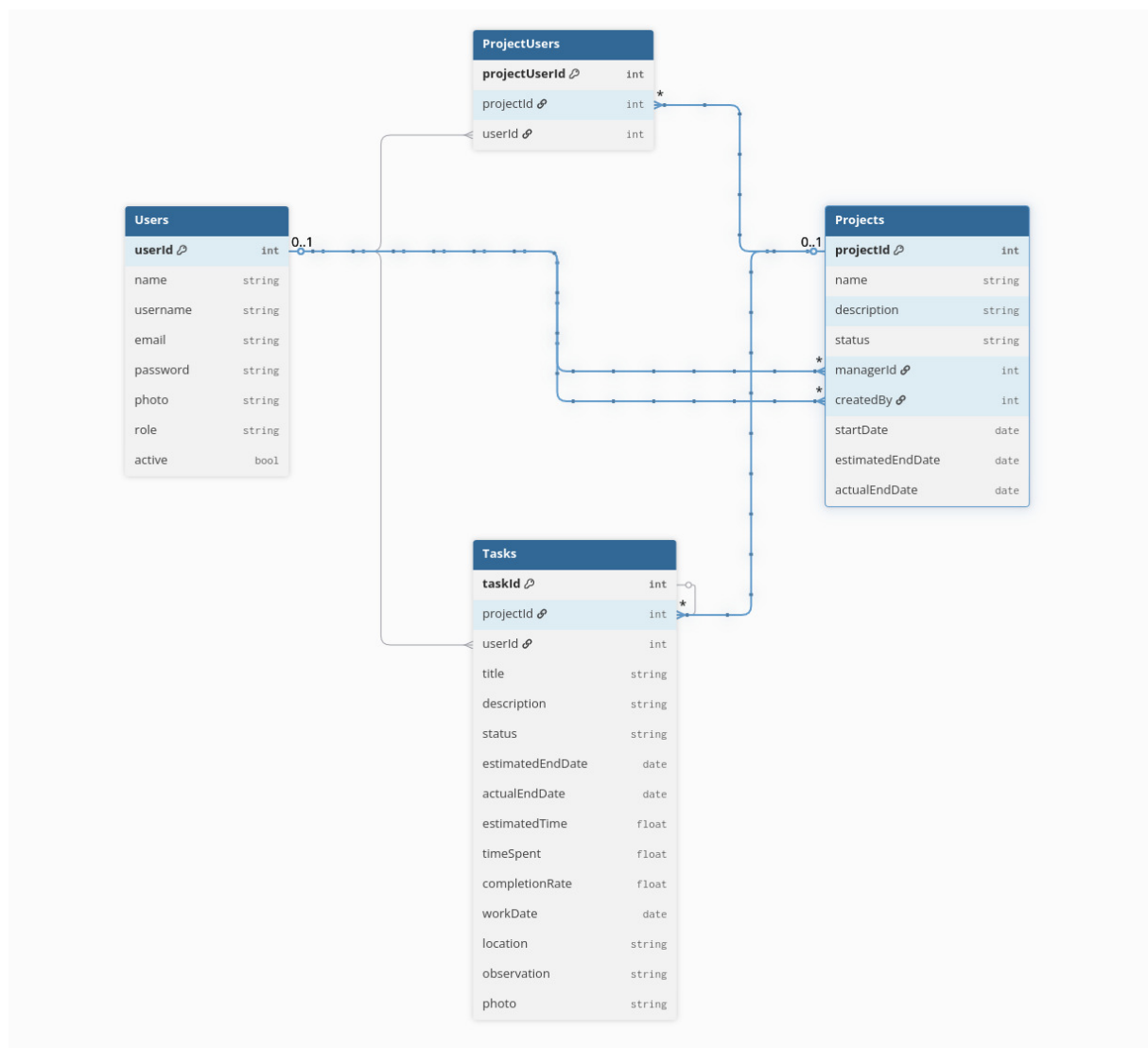


Figura 1: Modelo de dados da aplicação

6.1 Descrição das Principais Tabelas

Tabela 4: Descrição das principais tabelas da base de dados

Tabela	Descrição
Users	Guarda os dados dos utilizadores do sistema, incluindo credenciais, perfil e estado da conta.
Projects	Guarda os projetos existentes, incluindo informação de identificação, descrição, estado, datas e utilizadores responsáveis.
ProjectUsers	Representa a associação entre utilizadores e projetos, permitindo que vários utilizadores estejam associados ao mesmo projeto.
Tasks	Guarda as tarefas associadas aos projetos, incluindo estado, datas, progresso, tempo gasto, observações e fotografia.

7 UX e Design

7.1 Mockups

No âmbito da definição da experiência de utilização da aplicação, foram desenvolvidos mockups com o objetivo de representar visualmente os principais ecrãs do sistema. Estes mockups permitem validar a organização da interface, a navegação entre páginas e a coerência visual da aplicação antes da implementação final.

Os mockups do projeto COMMOV encontram-se disponíveis no Figma através do seguinte link:

Aceder aos mockups no Figma

Nesta fase, destacam-se especialmente a página de login e a página inicial da aplicação, por representarem dois momentos fundamentais da experiência do utilizador: o acesso ao sistema e a navegação principal após autenticação.

EN ▾

Welcome back

Enter your credentials to access your dashboard.

Email Address

 ricardo@projectflow.com

Password [Forgot Password?](#)

☐ Remember me for 30 days

Log In →

Don't have an account? [Create an account](#)

Figura 2: Mockup da página de login



Figura 3: Mockup da página inicial da aplicação

8 Gestão de Projeto

A gestão do projeto é uma componente importante no desenvolvimento, uma vez que permite organizar tarefas, acompanhar o progresso do trabalho e manter um registo das alterações efetuadas ao longo do tempo.

Nesta fase, a evidência disponível centra-se sobretudo na utilização do GitHub como plataforma de controlo de versões. A evidência relativa à utilização de uma plataforma de gestão de projeto, como Trello, ClickUp ou equivalente, será complementada posteriormente.

8.1 Evidência de Utilização de Git

O controlo de versões do projeto é realizado através do GitHub, permitindo acompanhar a evolução do código, consultar commits, gerir alterações e colaborar entre os elementos do grupo.

O repositório do projeto COMMOV encontra-se disponível através do seguinte link:

Aceder ao repositório COMMOV no GitHub

Para comprovar a atividade realizada no repositório, foram consultadas as estatísticas disponibilizadas pelo GitHub, nomeadamente as páginas *Pulse* e *Contributors*. Estas páginas permitem verificar a evolução dos commits, os autores envolvidos e a atividade registada num determinado período.

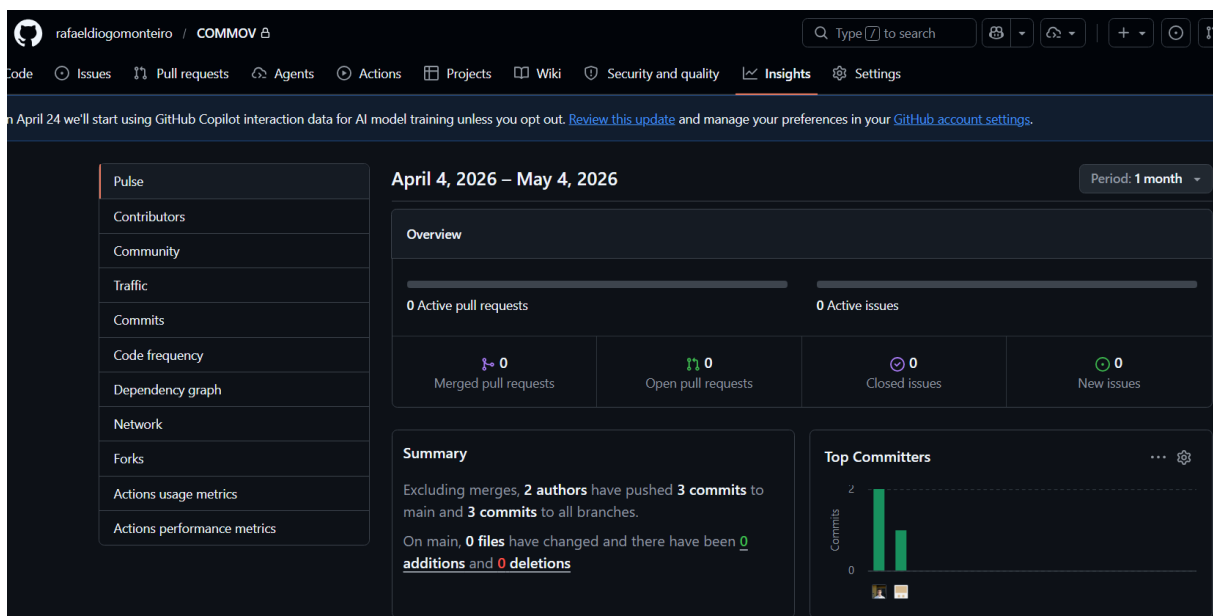


Figura 4: Resumo da atividade do repositório no GitHub

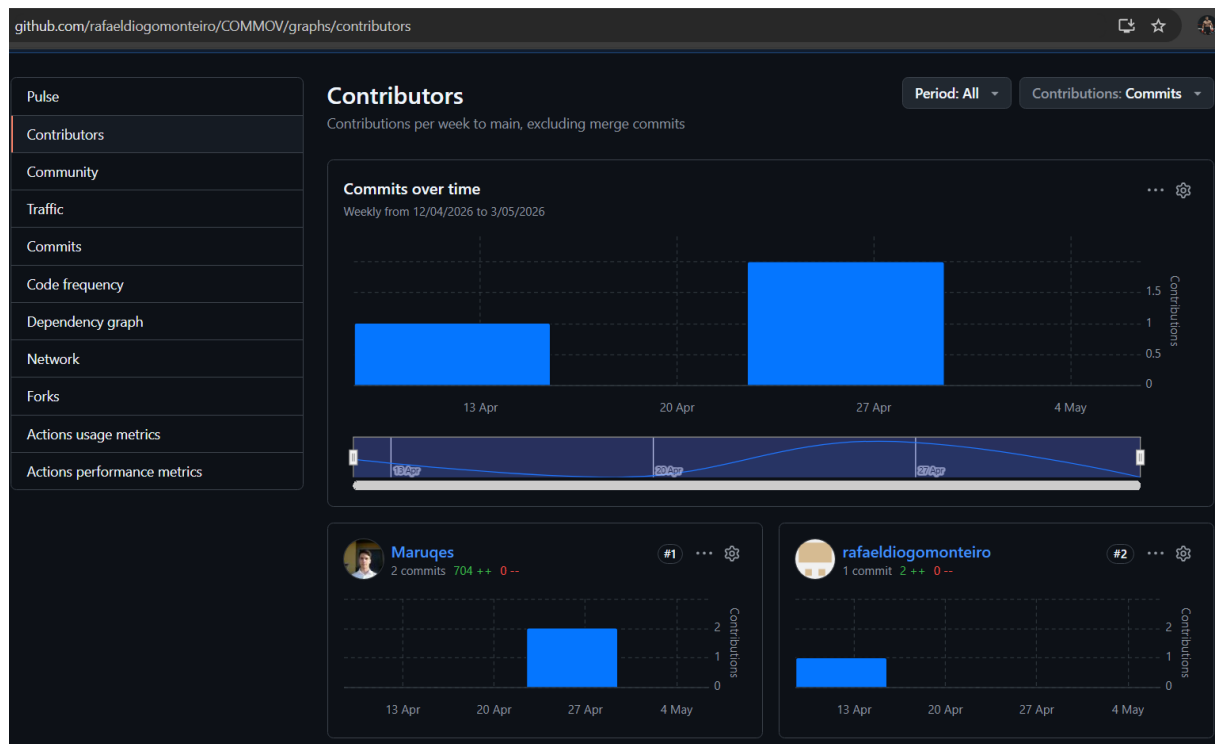


Figura 5: Contribuições por autor no repositório

8.2 Evidência de Utilização de uma Plataforma de Gestão de Projeto

Para apoiar a gestão do projeto COMMOV, foi criado um quadro no Trello organizado segundo o modelo Kanban. O quadro encontra-se dividido em três listas principais: **A fazer**, **Em andamento** e **Concluído**.

A lista **A fazer** inclui tarefas planeadas, como a implementação de endpoints, serviços e rotas. A lista **Em andamento** apresenta tarefas atualmente em desenvolvimento, como a autenticação de utilizadores e a atualização do relatório. Por fim, a lista **Concluído** reúne tarefas já realizadas, como a criação do repositório, a estrutura base do backend, a configuração da base de dados e a criação dos modelos principais.

Esta organização permite acompanhar o progresso do projeto de forma simples, visual e estruturada.

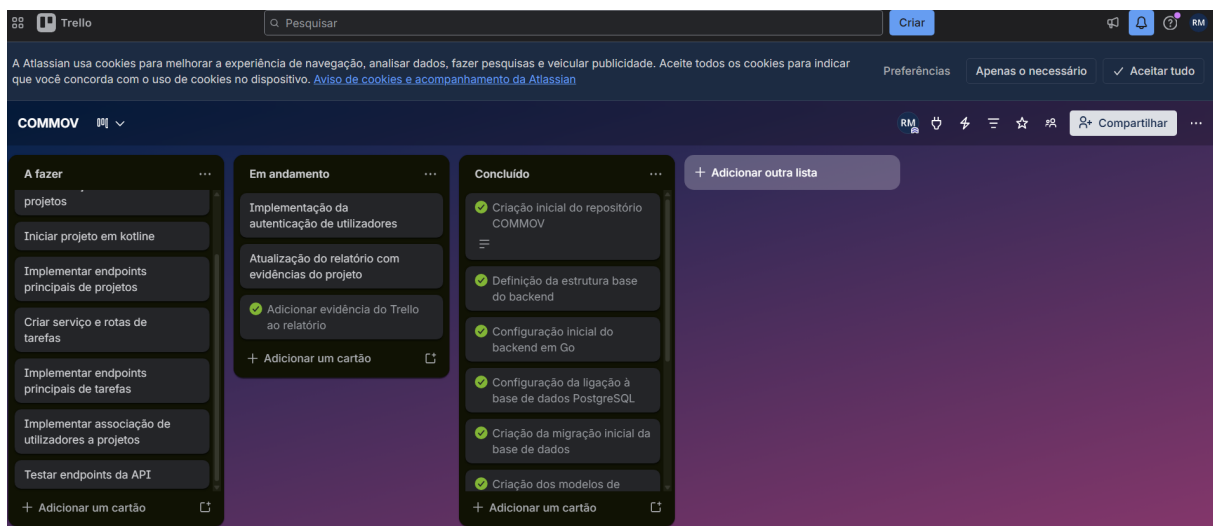


Figura 6: Quadro Trello utilizado para a gestão do projeto

9 Conclusão

O projeto encontra-se numa fase inicial, mas já apresenta uma base técnica consistente para evolução futura. O backend está estruturado por camadas, possui ligação a PostgreSQL, migrações iniciais, repositórios de dados e endpoints básicos para validação do funcionamento da API.

O modelo de dados já representa os principais conceitos necessários para a aplicação, nomeadamente utilizadores, projetos, tarefas e associações entre utilizadores e projetos. Esta estrutura permite avançar para a implementação das funcionalidades previstas nos requisitos funcionais.

Apesar de ainda existirem componentes importantes por desenvolver, como autenticação, autorização, frontend e endpoints completos para projetos e tarefas, o trabalho realizado até ao momento estabelece uma fundação adequada para continuar o desenvolvimento do sistema.